

Team Overview

Team Member	Module	Demo Section
Salman	Frontend + Authentication	0:00 – 1:30
Anika ★	Community + History + Content	1:30 – 3:00
Athoy	Admin Panel + Analytics	3:00 – 4:15

■ Chapter 1 — Your Demo Script (1:30 – 3:00)

You have exactly **90 seconds** on stage. Below is your step-by-step script with what to click, what to say, and what to highlight. Practice this until it feels natural.

		Emergency Blood Request
Step 1	0:00	■ Click Requests → Post Emergency Request. Fill in: Blood Type O–, Hospital Square Hospital, Units 3, Urgency Critical. Click Submit. ■ Say: "This instantly broadcasts to all compatible donors in the area."
		Search & Filter Requests
Step 2	0:25	■ Click Requests → Search Requests. Use the filter dropdowns — Blood Type, Urgency, Status. ■ Say: "Patients can search and filter all active requests by blood compatibility."
		■ Donor Map (WOW moment)
Step 3	0:40	■ Click Community → Donor Map. Show pins across Dhaka. Filter by blood type → pins change. Adjust radius → circle updates. Click a pin → donor popup appears. ■ Say: "Each pin is in the donor's actual neighborhood — we used OpenStreetMap Nominatim for geocoding."
		Status Tracker (6-Stage Pipeline)
Step 4	1:05	■ Click Requests → Status Tracker. Select a completed request. Show all 6 stages in green. ■ Say: "Open → Donors Notified → Matched → Contact Shared → Scheduled → Completed."
		Donation History
Step 5	1:20	■ Click Requests → Donation History. Show filters and the Recharts bar chart. ■ Say: "Full filterable audit trail with monthly trend visualization."

■ Chapter 2 — My Files & Their Locations

Every file below was written entirely by **Miskatul Afrin Anika**. The project follows the MVC pattern — frontend React components live in `client/src/components/` and backend logic lives in `server/`.

Frontend Files — `client/src/components/community/`

File	Purpose
<code>DonationHistoryPage.jsx</code>	F5 — Main history page with date, blood type, and status filters + paginated list
<code>DonationStatsCard.jsx</code>	F5 — Statistics card showing total donations, lives helped, and Recharts bar chart
<code>DonationStatsCard.css</code>	F5 — Styling for the stats card
<code>HistoryFilters.css</code>	F5 & F12 — Shared dark glassmorphism filter styles used by both history pages
<code>RequestHistoryPage.jsx</code>	F12 — Request history with expandable rows, status timeline, and hand-written CSV export (21KB — largest frontend file)
<code>LeaderboardPage.jsx</code>	F13 — Leaderboard table with All-Time/Monthly tabs and city filter dropdown
<code>Leaderboard.css</code>	F13 — Dark theme table styling with rank highlighting (gold/silver/bronze)
<code>BadgeShowcase.jsx</code>	F13 — Renders donation milestone badges on donor cards
<code>MilestonePopup.jsx</code>	F13 — Animated popup that appears when a donor earns a new badge
<code>EligibilityReminderCard.jsx</code>	F14 — Shows countdown of days until the donor is eligible to donate again
<code>EligibilityReminderCard.css</code>	F14 — Countdown display styling
<code>HealthTipsSection.jsx</code>	F14 — Health tips accordion/cards shown on donor profile page
<code>HealthTipsSection.css</code>	F14 — Tip card styling
<code>FeedbackForm.jsx</code>	F15 — Star rating (1–5) form with gratitude message and public/private toggle
<code>Feedback.css</code>	F15 — Animated star hover/click effects
<code>DonorFeedbackSection.jsx</code>	F15 — Displays public feedback ratings on a donor's profile page
<code>FAQPage.jsx</code>	F20 — Public FAQ page with category tabs and accordion expandable questions
<code>FAQ.css</code>	F20 — Premium dark theme FAQ styling (12KB)
<code>BloodCompatibilityChartPage.jsx</code>	F20 — Color-coded 8×8 grid showing which blood types can donate to which

Admin File — `client/src/components/admin/`

File	Purpose
<code>AdminContentEditor.jsx</code>	F20 — Admin CRUD interface to create, edit, delete, and reorder FAQ entries

Backend Files — server/

File	Purpose
<code>controllers/communityController.js</code>	Core backend logic: leaderboard aggregation, feedback CRUD, FAQ public endpoints
<code>controllers/adminController.js</code>	Admin-only FAQ management: <code>createFAQ()</code> , <code>updateFAQ()</code> , <code>deleteFAQ()</code>
<code>controllers/donorController.js</code>	<code>getDonationsHistory()</code> — filtered & paginated donation history query
<code>controllers/requestController.js</code>	<code>getMyRequests()</code> — filtered & paginated request history query
<code>models/Feedback.js</code>	Mongoose schema: <code>donorId</code> , <code>requestId</code> , <code>rating</code> , <code>message</code> , <code>allowPublic</code> , <code>timestamps</code>
<code>models/FAQ.js</code>	Mongoose schema: <code>question</code> , <code>answer</code> , <code>category</code> , <code>order</code> , <code>isActive</code>
<code>models/Donation.js</code>	Mongoose schema: <code>donor</code> , <code>recipient</code> , <code>blood type</code> , <code>date</code> , <code>status</code>
<code>jobs/reminderJob.js</code>	F14 — node-cron job: runs daily at 8:00 AM, finds eligible donors, sends reminder emails via Nodemailer

■ Chapter 3 — My 6 Features Explained Simply

Each feature is explained in plain language so you can describe it to faculty without jargon, followed by the key technical points.

F5 — Donation History & Statistics [Sprint 1]

What it does in plain words: When a donor logs in and clicks 'Donation History', they see a list of every blood donation they have ever made. They can filter this list by a date range (from/to), by blood type, or by status (Scheduled / Completed / Cancelled). A stats card at the top shows total donations and how many lives were helped. A bar chart shows how many donations were made each month.

How it works technically:

- The filter dropdowns in the React component build a URL query string like `?from=2025-01-01&to=2025-12-31&status=Completed`.
- The backend function `getDonationsHistory()` in `donorController.js` reads these query parameters and builds a MongoDB filter object dynamically.
- It uses `$gte` (greater than or equal) and `$lte` (less than or equal) for date range filtering.
- Results are paginated — 20 per page — to avoid loading thousands of records at once.
- The bar chart uses the **Recharts** library with the data grouped by month.

■ Key talking point: 'The filter builds a MongoDB query object dynamically based on whichever filters the user selected — unused filters are simply not added to the query.'

F12 — Request History & Records [Sprint 1]

What it does in plain words: Shows all blood requests the user has ever posted. Each row can be expanded by clicking it — this reveals full details and a visual timeline of all 6 stages (Open → Donors Notified → Matched → Contact Shared → Scheduled → Completed) with timestamps. There is also an 'Export CSV' button that downloads all filtered requests as a spreadsheet file.

The CSV export is hand-written (no library):

- Column headers are defined as an array.
- Each row of data is mapped to a comma-separated string.
- Special characters (commas, quotes, newlines) inside values are wrapped in double quotes, and internal double quotes are escaped as `""`.
- All rows are joined with newline characters.
- A `Blob` object with MIME type `text/csv` is created and a temporary link is programmatically clicked to trigger the browser download.

■ Why hand-written? The course rules require core algorithms to be written by the team — not imported from npm packages. This shows you understand the logic.

F13 — Donor Leaderboard & Milestone Badges [Sprint 2]

What it does in plain words: A public leaderboard showing the top donors ranked by how many times they have donated blood. You can switch between 'All Time' and 'This Month' tabs. There is a city dropdown to filter by location. Each donor's row shows their name, blood type, donation count, and any badges they have earned.

Badge milestones:

Badge	When Earned	Why
■ First Donation	Awarded at 1 donation	Encourages first-timers

■ Lifesaver	Awarded at 3 donations	Recognizes regular donors
■ Champion	Awarded at 5 donations	Highlights dedicated donors
■ Hero	Awarded at 10 donations	Top-tier recognition

Backend — MongoDB Aggregation Pipeline (hand-written ranking algorithm):

- `$match` — filters out suspended users; optionally filters by city.
- `$sort` — sorts by `donationCount` descending (highest first).
- `$limit` — takes only the top 50 results.
- `$project` — selects only needed fields to reduce data transfer.

For the monthly tab, the pipeline runs on the `Donation` collection and filters by `donationDate` within the current month.

F14 — Eligibility Reminders & Health Tips [Sprint 2]

What it does in plain words: Blood donors cannot donate again for a certain period after their last donation. This feature automatically emails them the day they become eligible again. It also shows health tips on the profile page (drink water, eat iron-rich foods, avoid heavy lifting, get enough sleep before donation).

How the cron job works:

- A **cron job** is a scheduled task that runs automatically at a set time — like an alarm clock for your server.
- The expression `'0 8 * * *'` means: minute 0, hour 8, every day.
- Every morning at 8:00 AM the server checks MongoDB for donors whose `nextEligibleDate` is today or earlier and `isAvailable` is false.
- For each eligible donor, it sends a reminder email through **Nodemailer** using Gmail SMTP.
- The library used is `node-cron`.

■ Analogy: Imagine the server sets a daily alarm. Every morning it checks a list and sends emails — like a hospital receptionist calling patients.

F15 — Donor-Requester Feedback System [Sprint 3]

What it does in plain words: After a blood request is completed, the person who made the request can rate the donor from 1 to 5 stars and write a thank-you message. They can choose to make this feedback public (visible on the donor's profile) or keep it private. Public feedback appears in a section on the donor's profile page.

Security checks (4 layers of validation):

1. **Frontend** — rating must be 1–5; submission only allowed for completed requests.
2. **Ownership check** — backend verifies the person submitting is the same person who made the request.
3. **Donor match check** — the donor being rated must be the one actually matched to the request (prevents false feedback).
4. **Duplicate check** — checks if feedback already exists for this request (one feedback per request).

F20 — FAQ & Content Management [Sprint 3]

What it does in plain words: A public Help/FAQ page with questions organized into category tabs (Eligibility, Blood Types, Preparation, After Donation). Each question expands like an accordion when clicked. There is also a blood type compatibility chart showing which blood type can donate to which. Admins have a separate editor where they can add new FAQ questions, edit existing ones, delete them, and set their order.

Blood Compatibility Chart: An 8×8 grid — rows are donors, columns are recipients. Each cell is color-coded: green = can donate, red = cannot donate. All 8 blood types are covered: A+, A−, B+, B−, AB+, AB−, O+, O−.

Admin CRUD operations:

- **Create** — `POST /api/admin/faqs` — Admin fills a form, FAQ is saved to MongoDB.
- **Read** — `GET /api/community/faqs` — Anyone can view active FAQs, sorted by order field.
- **Update** — `PUT /api/admin/faqs/:id` — Admin edits question, answer, category, or order.
- **Delete** — `DELETE /api/admin/faqs/:id` — Admin removes a FAQ entry.

■ Chapter 4 — My API Endpoints

Every API call your frontend makes goes through these endpoints. Know each one — faculty may ask you to trace a request.

Method	Endpoint	What It Does
GET	/api/donors/donations?from=&to;=&bloodType;=&status;=	Donation history with optional filters. Returns paginated list for logged-in donor.
GET	/api/requests/my?page=1&limit;=20	All blood requests made by the logged-in user. Paginated.
GET	/api/community/leaderboard?type=monthly&city;=Dhaka	Top donors ranked by donation count. type=alltime or monthly. city is optional filter.
GET	/api/community/faqs	All active FAQ entries sorted by order field. Public — no auth required.
GET	/api/community/feedback/donor/:id	All public feedback for a specific donor. Shown on their profile.
POST	/api/community/feedback	Submit a new feedback. Body: { donorId, requestId, rating, message, allowPublic }
POST	/api/admin/faqs	Admin creates a new FAQ. Body: { question, answer, category, order, isActive }
PUT	/api/admin/faqs/:id	Admin updates an existing FAQ by its MongoDB _id.
DELETE	/api/admin/faqs/:id	Admin deletes a FAQ entry permanently.

Every authenticated request includes the JWT token in the Authorization header:

```
Authorization: Bearer <your_jwt_token>
```

The token is stored in React's **AuthContext**. You access it in every component with: `const { token, user } = useAuth();`

■ Chapter 5 — Architecture & How Everything Connects

MVC Pattern

Layer	What It Means	Example from My Code
Model	Defines the shape of data in the database	Feedback.js — Schema with fields: donorId, rating, message, allowPublic
View	React components the user sees and interacts with	FeedbackForm.jsx — Renders the star rating form
Controller	Business logic — validates input, talks to the database, returns a response	communityController.js → submitFeedback() — Checks ownership, saves to MongoDB

Request Flow — What Happens When You Click Something

1	User Action	User clicks 'Export CSV' in RequestHistoryPage.jsx
2	React Component	Component maps the filtered data to CSV rows using hand-written algorithm
3	Blob Download	Creates a Blob → URL.createObjectURL() → triggers browser download — OR (for server-side requests like feedback submission) —
1	User Action	User submits the feedback form in FeedbackForm.jsx
2	Axios POST	axios.post('/api/community/feedback', data, { headers: {Authorization: Bearer token} })
3	Express Router	communityRoutes.js routes the request to communityController.submitFeedback()
4	Controller Logic	Validates ownership → checks donor match → checks for duplicates → saves to MongoDB
5	Response	Returns { success: true, feedback: {...} } as JSON back to React
6	UI Update	React re-renders to show success message and the new feedback on the profile

■ Chapter 6 — Viva Questions & Answers

These are the most likely questions faculty will ask based on your module. Each answer is written to be clear and confident — read them aloud until they feel natural.

Q1: What features did you build?

I built 6 features: Donation History with statistics and a bar chart, Request History with expandable rows and CSV export, a Donor Leaderboard with monthly/all-time tabs, an Eligibility Reminder system with a daily cron job, a Donor-Requester Feedback system with star ratings, and an FAQ page with admin content management.

Q2: What is the MVC architecture and how did you use it?

MVC stands for Model-View-Controller. The Model defines the data structure in MongoDB — for example, Feedback.js defines fields like donorId, rating, message, and allowPublic. The View is the React component the user sees — like FeedbackForm.jsx which renders the star rating UI. The Controller contains the business logic — communityController.js handles validation, database operations, and returns JSON responses.

Q3: What is a REST API? Give an example from your code.

REST is a set of rules for building web APIs using HTTP methods. GET retrieves data, POST creates data, PUT updates data, DELETE removes data. For example, when a user submits feedback, the frontend calls POST /api/community/feedback with the rating and message. The backend validates it, saves it to MongoDB, and returns a JSON response.

Q4: How does your leaderboard ranking work?

I use a MongoDB aggregation pipeline — a series of steps that transform the data. First, \$match filters out suspended users and optionally filters by city. Then \$sort orders donors by donationCount from highest to lowest. \$limit takes only the top 50. \$project selects only the fields we need to reduce data size. For the monthly tab, I run the same pipeline on the Donation collection filtered by donationDate within the current month.

Q5: Explain your CSV export. Why did you write it yourself?

The course rules require core algorithms to be hand-written, not imported from npm libraries. My CSV generator defines headers as an array, maps each data row to a comma-separated string, and handles edge cases: if a value contains commas, quotes, or newlines, it is wrapped in double quotes and internal double quotes are escaped as two double quotes. All rows are joined with newline characters. Then I create a Blob with MIME type text/csv, generate a download URL with URL.createObjectURL(), and programmatically click a temporary anchor element to trigger the file download.

Q6: How does the cron job work for eligibility reminders?

I use the node-cron library to schedule a function that runs daily at 8:00 AM — the cron expression is '0 8 * * *' meaning minute 0, hour 8, every day. The function queries MongoDB for donors whose nextEligibleDate is today or earlier and isAvailable is false. For each eligible donor it sends a reminder email via Nodemailer using Gmail SMTP. It is like setting a daily alarm on the server.

Q7: What is React Context API and how do you use it?

Context API lets you share data across React components without passing it through props at every level — this is called prop drilling. In our project, AuthContext.js provides the user object, the JWT token, and login/logout functions to the entire app. In my components I access it with: const { token, user } = useAuth(); Then I include the token in every API call header as: Authorization: Bearer token.

Q8: What is Mongoose and how did you define your schemas?

Mongoose is an Object Data Modeling library for MongoDB. It lets you define schemas that enforce structure on your database documents. For example, my Feedback schema requires a donorId, a requestId, a rating between 1 and 5, an optional message up to 500 characters, and an allowPublic boolean. The ref field in donorId enables the populate() method to automatically join data from the Donor collection — like a JOIN in SQL.

Q9: What is the difference between useState and useEffect?

useState declares a reactive variable — when you update it, React re-renders the component. For example: `const [rating, setRating] = useState(0)`; clicking a star calls `setRating(3)` and the component updates. `useEffect` runs code after the component renders — typically for API calls. The dependency array controls when it runs: `useEffect(() => { fetchLeaderboard(); }, [type, city])` runs whenever type or city changes.

Q10: How did you validate the feedback submission?

There are 4 layers. First, frontend validation — rating must be 1–5 and the request must be in Completed status. Second, ownership validation — the backend checks that the person submitting is the same user who created the request. Third, donor match validation — the donor being rated must be the one actually matched to the request, preventing someone from leaving fake feedback for a different donor. Fourth, duplicate prevention — we check if feedback already exists for this specific request to prevent multiple submissions.

Q11: What is responsive design and how did you implement it?

Responsive design means the UI adapts to different screen sizes — desktop, tablet, mobile. I implemented it using CSS media queries: `@media (max-width: 768px)` to adjust layouts for smaller screens. I used Flexbox for flexible containers, percentage-based widths instead of fixed pixels, and a mobile-first approach in the history filters and leaderboard table.

Q12: What is the difference between GET and POST?

GET retrieves data from the server — it is safe, has no body, and parameters go in the URL as query strings. For example: `GET /api/community/leaderboard?type=monthly` retrieves the monthly leaderboard. POST sends data to create something new — parameters go in the request body as JSON, not in the URL. For example: `POST /api/community/feedback` sends the rating and message to create a new feedback record.



Chapter 7 — My Hand-Written Algorithms

The course requires that core logic algorithms are written by the team — not imported from npm libraries. Here are the 4 algorithms I wrote from scratch.

■ CSV Export Algorithm

File: `RequestHistoryPage.jsx`

Generates a properly formatted .csv file directly in the browser without any library.

```
// 1. Define headers
const headers = ['Patient Name', 'Blood Type', 'Units', 'Hospital', 'Urgency', 'Status', 'Date'];

// 2. Handle special characters
const escapeCSV = (val) => {
  const str = String(val ?? '');
  if (str.includes(',') || str.includes('"') || str.includes('\n'))
    return `"${str.replace(/"/g, '"')}"`;
  return str;
};

// 3. Build CSV string and trigger download
const csvContent = [headers.join(','), ...rows.map(r => cols.map(escapeCSV).join(',')).join('\n')];
const blob = new Blob([csvContent], { type: 'text/csv' });
const url = URL.createObjectURL(blob);
// ... click <a> element to trigger download
```

■ Leaderboard Ranking Algorithm

File: `communityController.js` → `getLeaderboard()`

MongoDB aggregation pipeline to rank donors by donation count.

```
const pipeline = [
  { $match: { isSuspended: { $ne: true } } }, // exclude suspended users
  { $sort: { donationCount: -1 } }, // highest count first
  { $limit: 50 }, // top 50 only
  { $project: { name:1, bloodType:1, donationCount:1, badges:1 } }
];
if (city) pipeline[0].$match.city = new RegExp(`^${city}$`, 'i');
const leaderboard = await Donor.aggregate(pipeline);
```

■ Dynamic Filter Builder

File: `DonationHistoryPage.jsx` + `donorController.js`

Builds MongoDB filter objects dynamically from whichever query parameters are present.

```
// Backend (donorController.js)
const filter = { donorId: req.user._id };
if (req.query.bloodType) filter.bloodType = req.query.bloodType;
if (req.query.status) filter.status = req.query.status;
if (req.query.from || req.query.to) {
  filter.donationDate = {};
  if (req.query.from) filter.donationDate.$gte = new Date(req.query.from);
  if (req.query.to) filter.donationDate.$lte = new Date(req.query.to);
}
const donations = await Donation.find(filter).skip(skip).limit(20);
```

■ Date Formatter

File: Multiple files

Formats MongoDB ISO date strings into readable format without moment.js or date-fns.

```
// Example: '2025-03-15T08:30:00.000Z' → 'March 15, 2025'
const formatDate = (dateStr) => {
  const d = new Date(dateStr);
  return d.toLocaleDateString('en-US', {
    year: 'numeric', month: 'long', day: 'numeric'
  });
};
```

■ Chapter 8 — Git Workflow & My Branches

The team used a **feature-branch workflow**. Each feature was developed on its own branch, then merged into dev via a Pull Request. The final dev merge into main happened before the GitHub deadline (May 5, 2026 11:59 PM).

My Branch	Feature	Status
feature/anika-donation-history	F5 – Donation History & Statistics	■ Merged
feature/anika-request-history	F12 – Request History & CSV Export	■ Merged
feature/anika-leaderboard	F13 – Leaderboard & Badges	■ Merged
feature/anika-eligibility	F14 – Eligibility Reminders & Health Tips	■ Merged
feature/anika-feedback	F15 – Feedback System	■ Merged
feature/anika-faq-content	F20 – FAQ & Content Management	■ Merged

My Workflow Commands

```
# 1. Start from the latest dev branch
git checkout dev
git pull origin dev

# 2. Create a new feature branch
git checkout -b feature/anika-leaderboard

# 3. After coding – stage, commit, push
git add -A
git commit -m "feat: create LeaderboardPage with tabs and city filter"
git push origin feature/anika-leaderboard

# 4. Create a Pull Request on GitHub → merge into dev
# 5. Before deadline → dev merged into main
```

Commit Message Convention

Prefix	Used When	Example
feat:	Adding a new feature	feat: add FAQ accordion page with category tabs
fix:	Fixing a bug	fix: correct CSV escaping for fields with commas
style:	CSS changes only	style: update leaderboard table dark theme
docs:	Documentation updates	docs: add inline comments to reminderJob.js

Chapter 9 — My Full Contribution Summary

Total: **19 component files**, **6 CSS files**, **~1,200+ lines of frontend code**, plus backend controllers, models, and the cron job.

Area	What I Built	Files
Donation History (F5)	Filterable list with date range, blood type, and status filters;	DonationStatsCardPage.jsx DonationStatsCard.jsx DonationStatsCard.css HistoryFilters.css
Request History (F12)	Expandable rows with full details, 6-stage status timeline with RequestHistoryPage.jsx	RequestHistoryPage.jsx HistoryFilters.css
Leaderboard (F13)	All-time and monthly tabs, city filter dropdown, badge display, milestone popup animation, MongoDB aggregation pipeline	LeaderboardPage.jsx Leaderboard.css BadgeShowcase.jsx MilestonePopup.jsx
Eligibility System (F14)	node-cron daily email reminders, EligibilityReminderCard with EligibilityReminderCardHeader	EligibilityReminderCardHeader.jsx HealthTipsSection.jsx jobs/reminderJob.js
Feedback System (F15)	Star rating form (1–5) with gratitude message, public/private toggle, DonorFeedbackForm	FeedbackForm.jsx Feedback.css DonorFeedbackSection.jsx
FAQ & Content (F20)	Public accordion FAQ with category tabs, 8×8 blood compatibility chart, admin CRUD editor for FAQ management	FAQPage.jsx FAQ.css BloodCompatibilityChartPage.jsx AdminContentEditor.jsx

Technologies I Used

Technology	Purpose	Why Chosen
React.js	Frontend UI	Component-based, reusable, fast re-renders with state
Vanilla CSS	Styling	Full design control; dark glassmorphism needs custom gradients & backdrop-filter
Recharts	Bar charts	Lightweight React charting library — easy to integrate, responsive
Node.js + Express	Backend API	JavaScript full-stack; async I/O handles multiple requests efficiently
MongoDB + Mongoose	Database	Flexible document storage; aggregation pipeline for leaderboard ranking
node-cron	Scheduling	Runs daily eligibility reminder emails without needing external services
Nodemailer	Email	Send reminder emails via Gmail SMTP — free, simple setup
Axios	HTTP calls	Promise-based, clean syntax, easy to attach Authorization headers

■ Chapter 10 — Quick Reference Cheat Sheet

Keep this page open during your viva. It has everything you might need to recall quickly.

Key File Locations At a Glance

If faculty asks about...	Open this file
Donation history filters	client/src/components/community/DonationHistoryPage.jsx
Monthly bar chart	client/src/components/community/DonationStatsCard.jsx
CSV export code	client/src/components/community/RequestHistoryPage.jsx
Leaderboard ranking	server/controllers/communityController.js → getLeaderboard()
Badge milestone logic	client/src/components/community/BadgeShowcase.jsx
Cron job / reminder emails	server/jobs/reminderJob.js
Health tips section	client/src/components/community/HealthTipsSection.jsx
Star rating feedback form	client/src/components/community/FeedbackForm.jsx
Feedback validation (backend)	server/controllers/communityController.js → submitFeedback()
Feedback schema (MongoDB)	server/models/Feedback.js
FAQ public page	client/src/components/community/FAQPage.jsx
Blood compatibility chart	client/src/components/community/BloodCompatibilityChartPage.jsx
Admin FAQ editor	client/src/components/admin/AdminContentEditor.jsx
FAQ schema	server/models/FAQ.js

Impress-Faculty Talking Points

★ Hand-written algorithms

CSV export and leaderboard ranking are written from scratch — no npm libraries — as required by the course. This shows I understand the underlying logic.

★ 4-layer feedback validation

Frontend, ownership, donor-match, and duplicate-prevention checks. Security was not an afterthought — it was designed in from the start.

★ MongoDB aggregation pipeline

The leaderboard uses a proper pipeline with `$match`, `$sort`, `$limit`, `$project` instead of loading all donors and sorting in JavaScript.

★ Scheduled server task

The node-cron eligibility reminder runs automatically every day at 8:00 AM — no manual intervention needed.

★ Dark glassmorphism design

All 6 CSS files use backdrop-filter blur, custom gradients, and consistent dark theme — built with vanilla CSS for maximum control.

★ 19 React components

Each component has a single responsibility — following React best practices with clean separation of concerns.

Best of luck on your viva, Anika! You built 6 complete features — know them, own them. ■

CSE470 — Software Engineering | Spring 2026